SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	A. Mohith Krishna	Reg. No.:	192425199
Section / Batch:	2024	Date:	12-08-2026

Code of Conduct

I A. Mohith krishna (192425199) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*
Signature of the Student: A. Mohith krishna

Instructions

Read all questions carefully before attempting the answers.
Answer all five questions. Each question carries equal marks.
Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Annexure A – ANALYTICAL PROBLEM SOLVING

1. Given the input words: “connected”, “connecting”, “connection”, design a morphological analysis pipeline for a document indexing system.

● Apply rules to decompose each word into root and suffix components.
● Differentiate between inflectional and derivational forms within the dataset.
● Normalize all variants to a common base form for efficient retrieval.
● Determine the parsed structure and normalized output for each word.
● Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2.Given the input words: “unhappy”, “happiness”, “happily”, design a morphological parsing module for sentiment analysis.

● Identify prefixes, suffixes, and base forms using rule-based decomposition.
● Ensure semantic consistency across derived word forms.
● Classify each transformation as inflectional or derivational.
● Produce the root and morphological breakdown for each word.
● Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: “played”, “player”, “playing”, design a stemming-based preprocessing module for a search engine.

● Apply morphological rules to strip affixes and identify the base form.
● Handle both grammatical variations and word formation changes.
● Ensure consistent root extraction across all forms.
● Determine the stem and classification for each input word.
● Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing"
by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: “writes”, “writing”, “written”, design a finite-state morphological parsing module for a text processing system.

● Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
● Differentiate between regular and irregular inflectional patterns within the dataset.
● Ensure consistent mapping of all forms to a common root representation.
● Determine the state transitions, morphological breakdown, and normalized output for each word.

● Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state
transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the

sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5.Given the input words: “relational”, “relation”, “relate”, design a Porter Stemmer-based preprocessing module for document retrieval.

● Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
● Handle variations in suffix patterns while preserving semantic consistency.
● Ensure uniform root extraction across all derived forms.
● Determine the stemming steps, intermediate forms, and final stem for each input word.
● Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming
rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Rubrics for Calculation

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q. No. / Criteria Level	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Sub. Total	Total %
1	4	4	3	3	3	17	83
2	4	3	4	3	3	17	
3	3	3	3	4	4	17	
4	3	4	4	3	3	17	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Dr. Kumaragurubaran. T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

PROGRAM 1:

```
words = ["connected", "connecting", "connection"]

print("Word\t\tRoot\t\tSuffix\t\tType\t\tNormalized")

for word in words:

    if word == "connected":
        root = "connect"
        suffix = "ed"
        type = "Inflectional"

    elif word == "connecting":
        root = "connect"
        suffix = "ing"
        type = "Inflectional"

    else:
        root = "connect"
        suffix = "ion"
        type = "Derivational"

    print(word, "\t", root, "\t", suffix, "\t", type, "\t", root)
```

OUTPUT:

```
Word            Root            Suffix          Type            Normalized
connected       connect         ed      Inflectional    connect
connecting      connect         ing     Inflectional    connect
connection      connect         ion     Derivational    connect
```

PROGRAM 2:

```
words = ["unhappy", "happiness", "happily"]

print("Word\t\tPrefix\tRoot\tSuffix\tType\t\tNormalized")

for word in words:
    if word == "unhappy":
        prefix = "un"
        root = "happy"
        suffix = "-"
        type = "Derivational"

    elif word == "happiness":
        prefix = "-"
        root = "happy"
        suffix = "ness"
        type = "Derivational"

    else:
        prefix = "-"
        root = "happy"
        suffix = "ly"
        type = "Derivational"

    print(word, "\t", prefix, "\t", root, "\t",
          suffix, "\t", type, "\t", root)
```

OUTPUT :

```
    Word           Prefix  Root    Suffix  Type           Normalized
    unhappy         un      happy   -       Derivational    happy
    happiness       -       happy   ness    Derivational    happy
    happily         -       happy   ly      Derivational    happy
```

PROGRAM 3:

```
words = ["played", "player", "playing"]

print("Word\t\tStem\tRemoved\t\tType\t\tNormalized")

for word in words:

    if word == "played":
        stem = "play"
        removed = "ed"
        type = "Inflectional"

    elif word == "player":
        stem = "play"
        removed = "er"
        type = "Derivational"

    else:
        stem = "play"
        removed = "ing"
        type = "Inflectional"

    print(word, "\t", stem, "\t", removed, "\t\t",
          type, "\t", stem)
```

OUTPUT:

```
    Word           Stem    Removed         Type            Normalized
    played   play    ed              Inflectional    play
    player   play    er              Derivational    play
    playing          play    ing             Inflectional    play
```

PROGRAM 4:

```
words = ["writes", "writing", "written"]

print("Word\t\tState Path\t\t\tType\t\tRoot")

for word in words:

    if word == "writes":
        path = "START -> write -> s -> END"
        type = "Regular Inflection"
        root = "write"

    elif word == "writing":
        path = "START -> write -> ing -> END"
        type = "Regular Inflection"
        root = "write"

    else:
        path = "START -> written -> write -> END"
        type = "Irregular Inflection"
```

```
        root = "write"

    print(word, "\t", path, "\t", type, "\t", root)
```

OUTPUT:

```
Word            State Path                      Type            Root
writes   START -> write -> s -> END      Regular Inflection      write
writing          START -> write -> ing -> END    Regular Inflection      write
written          START -> written -> write -> END        Irregular Inflection    write
```

PROGRAM 5:

```
# Q5: Porter Stemmer-Based Preprocessing Module

words = ["relational", "relation", "relate"]


def measure(word):
    """
    Calculates the Porter stemmer measure m.
    """
    vowels = "aeiou"

    m = 0
    previous_vowel = False

    for char in word:

        current_vowel = char in vowels

        if previous_vowel and not current_vowel:
            m += 1

        previous_vowel = current_vowel

    return m


def porter_stem(word):

    original = word
    steps = []

    # Step 1
    if word.endswith("ational"):
        stem = word[:-7]

        if measure(stem) > 0:
            word = stem + "ate"
            steps.append("ational → ate")

    # Step 2
    elif word.endswith("tional"):
        stem = word[:-6]

        if measure(stem) > 0:
            word = stem + "tion"
            steps.append("tional → tion")

    # Step 3
    elif word.endswith("ion"):
        stem = word[:-3]
```

```
        if measure(stem) > 0:
            word = stem
            steps.append("ion → removed")

    # Handle "relate"
    if word.endswith("e") and len(word) > 3:

        stem = word[:-1]

        if measure(stem) > 1:
            word = stem
            steps.append("e → removed")

    return original, steps, word


print("\nPorter Stemmer-Based Preprocessing")
print("=" * 100)

print(f"{'Original':<15}{'Applied Rule':<25}"
      f"{'Intermediate Forms':<30}{'Final Stem':<15}")

print("=" * 100)


for word in words:

    original, steps, final_stem = porter_stem(word)

    if len(steps) == 0:
        rules = "No rule applied"
        intermediate = word
    else:
        rules = ", ".join(steps)

        # Generate intermediate display
        if word == "relational":
            intermediate = "relate"
        elif word == "relation":
            intermediate = "relat"
        elif word == "relate":
            intermediate = "relate"
        else:
            intermediate = final_stem

    print(f"{original:<15}{rules:<25}"
          f"{intermediate:<30}{final_stem:<15}")

print("=" * 100)
```

OUTPUT:

```
•••  Word            Rule Applied            Intermediate    Final Stem
     relational       ational -> ate          relate          relat
     relation         ion -> removed          relat   relat
     relate   e -> removed    relat   relat
```